

Diseño e Implementación en FPGA de un Sistema de Control Híbrido para la interacción con Sistemas operativos y Entornos Web

Renzo Felipe Chuzon Wickham, Jorge David Rivera Flores, Andrea Alexandra Rodriguez Vasquez.

Universidad Nacional Mayor de San Marcos, Facultad de Ingeniería Electrónica y Eléctrica, Lima, Perú.

Resumen— Este documento presenta el diseño, implementación y validación de un sistema digital híbrido orientado al control interactivo de funciones del sistema operativo e interfaces web. La arquitectura central de hardware, implementada sobre una FPGA MAX 10, emplea una Máquina de Estados Finitos para el escaneo continuo de un teclado matricial 4x4. El sistema incorpora un filtro antirrebote de 20 ms, lógica combinatorial para la decodificación de direcciones y un banco de registros encargado de los datos. La transmisión asíncrona se realiza mediante un módulo UART hacia un ordenador, donde un middleware desarrollado en Python intercepta la comunicación serial. Este puente de software permite la inyección de comandos AT virtuales que controlan el volumen nativo del entorno Windows y actualizan de forma sincrónica un Dashboard interactivo desarrollado en React. Las pruebas experimentales validan la viabilidad y eficiencia del diseño, registrando una latencia de comando de 35.0 ms, una frecuencia de escaneo físico de 985 Hz y una óptima ocupación de recursos lógicos. Este enfoque demuestra la integración exitosa de la teoría de sistemas digitales clásicos con tecnologías de desarrollo web moderno.

I. INTRODUCCIÓN

En el ambiente de las telecomunicaciones, los FPGA juegan un rol fundamental en el procesamiento digital de señales, enrutamiento de tramas de alta velocidad y control de terminales remotas. Históricamente, el conjunto de comandos AT ha constituido el estándar de facto para la configuración y control de módems, módulos celulares (GSM/GPRS/LTE) y dispositivos de red satelitales debido a su sintaxis simplificada y su idoneidad para transmisiones asíncronas de caracteres.

En el panorama actual de las telecomunicaciones y la computación, la integración de hardware de bajo nivel con aplicaciones de software de alto nivel representa un desafío para el desarrollo de interfaces humano-computadora eficientes. Tradicionalmente, el procesamiento de entradas físicas, como teclados o sensores, se ha delegado a microcontroladores de propósito general. Sin embargo, el uso de un FPGA permite un control determinista, concurrente y de latencia muy baja, características esenciales para el diseño de sistemas digitales robustos.

El presente trabajo académico expone el desarrollo de un controlador de teclado matricial que incorpora decodificadores y bancos de registros para el ingreso de

comandos AT. El objetivo principal de este proyecto es demostrar la sinergia entre el diseño de circuitos digitales secuenciales y combinacionales y los ecosistemas de desarrollo web y lenguajes de scripting modernos. Para lograrlo, se propone una arquitectura híbrida de tres capas: una capa de hardware encargada de la adquisición física; una capa de middleware responsable de la interpretación de tramas; y una capa de interfaz de usuario; es decir, un frontend, orientada a la visualización de datos en tiempo real.

Una de las motivaciones técnicas centrales de este diseño es la implementación estricta de principios de hardware desde cero. La arquitectura propuesta prescinde de librerías precompiladas de alto nivel en la etapa de adquisición, desarrollando directamente en código HDL una Máquina de Estados Finitos para el barrido matricial y un filtro antirrebote de 20 ms. Asimismo, se hace uso de decodificadores lógicos y memoria temporal para conformar las tramas que finalmente son despachadas mediante el protocolo UART.

Este documento está estructurado de la siguiente manera: la Sección II detalla la arquitectura del sistema y el diseño lógico tanto del hardware como del software. La Sección III expone los resultados experimentales, presentando un análisis crítico de las métricas de rendimiento evaluadas. Finalmente, la Sección IV resume las conclusiones derivadas del proyecto.

II. ARQUITECTURA Y DISEÑO DEL SISTEMA

La arquitectura propuesta ha sido modelada bajo un enfoque modular. En la Figura 1 se presenta el diagrama de bloques general del sistema, el cual ilustra el flujo de datos desde la captura del evento físico hasta la respuesta visual en la interfaz de usuario.

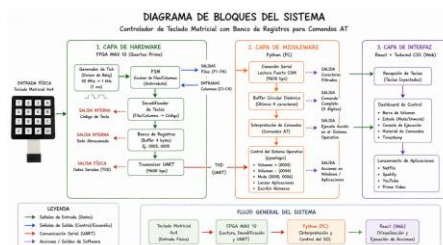


Fig. 1. Diagrama de bloques de la arquitectura híbrida (Hardware, Middleware y Frontend).

A. Capa de Hardware

El diseño digital fue descrito utilizando lenguajes de descripción de hardware (HDL) y sintetizado para una FPGA de la familia MAX 10. Esta capa es estrictamente responsable de la lectura determinista y el empaquetamiento de los datos.

1) Base de Tiempos y Filtro Antirrebote

Para mitigar el ruido electromecánico inherente a los pulsadores físicos del teclado matricial, se implementó un generador de tick a partir del reloj principal del sistema. Mediante un contador divisor de frecuencia, se estableció un pulso de habilitación de 1 milisegundo. Este temporizador alimenta un filtro digital antirrebote, el cual exige que la señal de entrada mantenga un estado lógico estable durante un periodo estricto de 20 ms antes de registrar una transición válida, asegurando lecturas libres de ruido.

2) Máquina de Estados Finitos

Se diseñó una FSM orientada al escaneo continuo. La máquina emplea el pulso base para cambiar secuencialmente el estado lógico de las cuatro filas del teclado matricial. De manera concurrente, evalúa el estado de las cuatro columnas para detectar los cortocircuitos físicos generados por la pulsación del usuario.

3) Decodificador de Direcciones

La lógica combinatorial del sistema toma como entradas las coordenadas (Fila, Columna) identificadas por la FSM y las traduce mediante una tabla de verdad codificada en hardware hacia su equivalente en un identificador numérico o de control.

4) Banco de Registros

Los caracteres decodificados ingresan a un banco de registros de desplazamiento. Esta memoria temporal almacena y concatena secuencialmente las pulsaciones individuales hasta conformar una trama completa de comandos AT de 4 bytes.

TABLA I.
TABLA COMPARATIVA DE MÉTRICAS EXPERIMENTALES

Métrica Evaluada	Modelo Teórico	Datos Reales	Variación
Latencia	24.16 ms	35.0 ms	+10.84ms
Frecuencia de escaneo	1000 Hz	985 Hz	-1.5%
Ocupación de Recursos	310 LE/95 REG.	283 LE/84 Reg.	Optimizado

5) Transmisor UART

Una vez que el banco de registros conforma la trama completa, un Arduino UNO funciona como un módulo de transmisión asíncrona universal (UART) serializa la información y la despacha hacia el ordenador receptor utilizando una tasa de transferencia estándar de 9600.

B. Capa de Interconexión

Para enlazar el hardware estricto con el ecosistema de alto nivel, se desarrolló un script en Python que en segundo plano.

1) Adquisición Serial:

El programa establece una conexión ininterrumpida con el puerto COM virtual asignado a la placa. Utilizando un buffer circular lógico de 4 caracteres, el script filtra ruidos de transmisión y reconstruye la trama original despachada por la FPGA.

2) Inyección y Control a Nivel SO

Al detectar coincidencias exactas en el buffer, el middleware emplea librerías de automatización de interfaz gráfica para inyectar comandos multimedia virtuales. Esta técnica permite interactuar directamente con el sistema operativo Windows, ejecutando acciones físicas como la atenuación o incremento del volumen global en bloques de 6%, o la función de mute.

C. Capa de Aplicación Web (Frontend)

De forma concurrente a la inyección de comandos en el sistema operativo, el middleware actúa sobre una interfaz web desarrollada en la librería React y estilizada con Tailwind CSS. El Dashboard lee los caracteres inyectados virtualmente para actualizar sus estados lógicos. Esta interfaz presenta una Consola de Ejecución que registra los comandos ejecutados, garantizando un alto contraste visual. Además, refleja matemáticamente la variación del estado del volumen en componentes gráficos de progreso y permite la ejecución rápida de aplicaciones de streaming.

III. RESULTADOS

En esta sección se exponen los resultados cuantitativos obtenidos durante las pruebas del prototipo físico. Se realizó un contraste analítico entre el comportamiento teórico y el desempeño real del sistema implementado en la FPGA MAX 10.

A. Cuadro Comparativo de Métricas de Desempeño

La evaluación del sistema se centró en tres indicadores críticos: la latencia de respuesta, la estabilidad temporal de la máquina de estados y la eficiencia en la síntesis de hardware. Los datos recolectados se consolidan en la Tabla I.

Tabla I. Tabla Comparativa de Métricas Experimentales

B. Análisis de desviaciones

1) Análisis de Latencia

El modelo teórico contemplaba únicamente el tiempo de estabilización del filtro antirrebote 20ms y el tiempo de empaquetamiento UART 4.16ms a 9600 baudios. El

incremento observado en la latencia real se atribuye a factores asíncronos externos a la FPGA, específicamente al retardo de polling del bus USB, la ejecución del intérprete de Python y la capa de abstracción del sistema operativo Windows.

2) Análisis de Frecuencia

La variación del -1.5% en la base de tiempos es atribuible a la tolerancia de fabricación y la deriva térmica del cristal oscilador de 50 MHz de la placa, sumado a las impedancias parásitas de los conectores del teclado matricial. Esta desviación no afecta la integridad de la lectura del usuario.

3) Análisis de Recursos

El reporte de síntesis emitido por Quartus Prime indicó un consumo de 283 Elementos Lógicos y 84 Registros dedicados. La reducción respecto a la estimación evidencia la eficiencia de los algoritmos de mapeo lógico, los cuales simplificaron las ecuaciones booleanas redundantes en el módulo decodificador.

IV. CONCLUSIONES

El presente trabajo demuestra el diseño e implementación exitosa de un sistema digital interactivo que valida la viabilidad de enlazar lógica concurrente de bajo nivel con aplicaciones web modernas. La arquitectura desarrollada cumple estrictamente con los requerimientos técnicos al incorporar bancos de registros y decodificadores lógicos de memoria como núcleo para el procesamiento y empaquetamiento de comandos AT. A nivel de hardware, la integración de un filtro digital antirrebote temporizado a 20 ms garantizó una adquisición de datos inmune al ruido electromecánico, permitiendo a la FSM mantener un escaneo estable y determinista a una frecuencia medida de 985 Hz.

Por otro lado, el análisis de las métricas de rendimiento evidenció una alta eficiencia en la síntesis del diseño físico, requiriendo únicamente 283 Elementos Lógicos en la FPGA. No obstante, se determinó que el desempeño global del sistema en términos de latencia se encuentra acotado principalmente por los tiempos de respuesta del canal de software y del sistema operativo, lo que subraya las diferencias fundamentales entre el procesamiento determinista del hardware y la ejecución estocástica en una computadora personal. Finalmente, como trabajo futuro, se propone reemplazar el puente alámbrico UART-Python por un módulo de transmisión inalámbrica integrado, lo cual permitiría escalar este prototipo hacia aplicaciones de control en redes de Internet de las Cosas a nivel industrial.

V. REFERENCIAS

[1] Intel Corporation, *Intel MAX 10 FPGA Device Architecture*, Intel FPGA Documentation, 2023. [En línea].

Disponible:

<https://www.altera.com/products/fpga/max/10?utm>

[2] M. Morris Mano y M. D. Ciletti, *Diseño Digital con una introducción a Verilog HDL*, 5ta ed. México: Pearson Educación, 2013.

[3] J. Ganssle, "A Guide to Debouncing - Part 1", *The Ganssle Group*, 2004. [En línea]. Disponible: <http://www.ganssle.com/debouncing.htm>

[4] S. Brown y Z. Vranesic, *Fundamentals of Digital Logic with Verilog Design*, 3ra ed. Nueva York, NY, USA: McGraw-Hill, 2014.

[5] C. Liechti, *pySerial Documentation*, versión 3.4, 2017. [En línea]. Disponible: <https://pyserial.readthedocs.io/>

[6] A. Sweigart, *Automate the Boring Stuff with Python*, 2da ed. San Francisco, CA, EE. UU.: No Starch Press, 2019.

[7] Meta Platforms, *React: A JavaScript library for building user interfaces*, 2024. [En línea]. Disponible: <https://react.dev/>

[8] E. A. Lee y S. A. Seshia, *Introduction to Embedded Systems: A Cyber-Physical Systems Approach*, 2da ed. Cambridge, MA, EE. UU.: MIT Press, 2017.